

Isolon: Keeping Isolation Claims Honest in a Bare-Metal Type-1 Hypervisor

Anonymous Author(s)
Submission Id: eurosys27-fall-anon

Abstract

Attaching a machine-checked isolation theorem to a Type-1 hypervisor that ships as a UEFI binary creates a labelling problem the verification literature usually hides: the proved artifact and the binary that leaves the compiler are not the same object. This paper reports Isolon, a clean-slate Type-1 hypervisor written in Rust, and the discipline we used to keep that gap from becoming a false claim.

The instrument is a four-level maturity ladder (documented invariants, runtime-enforced, spec-written, proof-complete). We argue that verification papers should state which column a claim lives in. What Isolon actually discharges is *exclusivity among guests* in a host-only Verus ghost model: two finite maps forming a bijection, preserved under map, unmap, EPT-violation handling, and a page-transfer primitive. Hypervisor separation is a modelling assumption, not a theorem about hypervisor page tables. The property is about EPT *mappings*, not about hypervisor *access*: virtio emulation reads and writes guest-owned virtqueue frames without a second EPT owner. There is no device passthrough; guest DMA is emulation-mediated. The live engine is a 512-slot 4K table plus a 16-slot range registry; Linux bring-up covers a 512 MiB identity window with range claims and 2M hardware leaves, not 131,072 ghost 4K keys. SMT found no mapping bug in the executable; we say so.

On a PowerEdge R640 the EFI reaches an unmodified Linux 6.12 shell and, on the same boot, virtio-blk, virtio-net, dual-vCPU SMP, and four-VM probes, then serves host-NIC HTTP. Nested VT-x completed a distro install-to-disk; that nested host is not the R640. Five lessons generalize: pin the prover, do not host SMT on UEFI, do not poke third-party firmware, firmware TCP is not a management plane, and nested is not iron.

CCS Concepts: • Software and its engineering → Software verification; Virtual machines; • Security and privacy → Logic and verification.

Keywords: formal verification, Type-1 hypervisor, EPT, Verus, maturity ladder, experience

ACM Reference Format:

Anonymous Author(s). 2027. Isolon: Keeping Isolation Claims Honest in a Bare-Metal Type-1 Hypervisor. In *Proceedings of European Conference on Computer Systems (EuroSys '27)*. ACM, New York, NY, USA, 10 pages.

1 Introduction

A hypervisor’s job is to make one computer look like several, without letting those several computers look at each other. On modern Intel servers the mechanism that actually enforces that is Extended Page Tables (EPT) [10]. If EPT is wrong, two guests share a frame, or a guest sees hypervisor memory, and no amount of management-API polish repairs it.

Commercial Type-1 stacks—ESXi, Xen [2], KVM [11]—are large, mature, and tested. They are not accompanied by a machine-checked isolation theorem for the frames they hand to guests. Research systems that *are* proved (seL4 [12, 13], CertiKOS [7], Komodo [6]) target microkernels, certified kernels, or enclaves. They are not a single UEFI binary aimed at the PowerEdge fleet already in the rack.

Isolon is a clean-slate Type-1 hypervisor written in Rust. It compiles to one `.efi` and boots on Dell PowerEdge R640-class machines we serial-log. Formal verification is an architectural constraint, not a post-hoc whitepaper. We picked Verus [15, 16] for SMT proofs of ghost state and Kani [5, 14] for bounded checking of unsafe. The interesting problem is not that a bijection between two maps can be proved. It is that a project shipping a hypervisor will be tempted to put that bijection in the abstract and then spend the rest of the paper taking it back. We treated that temptation as the thing to study.

The proved artifact, after every disclaimer a careful author would write, is exclusivity among guests in a host-only ghost model. Live executable EPT is a bounded ownership registry. Device emulation, HTTP, the SPA, ISO handling, and guest firmware are outside the Proven Core. Claiming “the hypervisor is proved” would be proving the wrong thing [12]: a convenient proxy. CI reports 80 verified Verus functions with 0 errors and no admit on the exclusivity path; that count is a replayable obligation tally, not a difficulty metric, and one of the 80 is a CI alias of another.

This paper makes four contributions:

1. **A maturity ladder as a labelling instrument**, not only as internal process. L0 documented invariants, L1 runtime asserts plus Kani, L2 spec-written, L3 proof-complete. We propose that verification papers state which column each claim occupies. Isolon’s ghost exclusivity is L3; live EPT is L2; the R640 runtime is L1.
2. **Five lessons that generalized** while shipping the EFI (Section 7): pin the prover, do not host SMT on a UEFI target, do not poke third-party firmware control state,

firmware TCP is not a management plane, and nested VT-x is not the production server. Table 6 archives the negative results behind those lessons.

3. **A guest-exclusivity ghost model** as the artifact we could actually prove. Two finite maps, a bijection, preservation under map, unmap, violation dispositions, and page transfer. Hypervisor “belongs to neither” is true by construction of the registry and is listed with the SDM and the SMT stack as an assumption. Modelling choices that were not free: 2M/1G span encoding, the emulate/reject/claim trichotomy, and NUMA affinity coupling.
4. **A split, measured evaluation.** Tree-wide line counts and the shipped .efi size against a 15 MB target. On one PowerEdge R640, COM2 records Linux 6.12 to a shell, virtio-blk, virtio-net, dual-vCPU SMP, four-VM probes, host-NIC HTTP, and El Torito CD EFL. Nested VT-x completed a distro sysinstall. We do not collapse those columns, and we do not invent a KVM cycle comparison we have not run.

Isolon is not seL4 for hypervisors and is not a verified KVM. It is a hypervisor we boot on iron, a ghost model we discharge, and a ladder that stops those two facts from being fused in a sentence.

2 Threat Model and Non-Goals

2.1 Why memory isolation is the headline

A Type-1 hypervisor is the TCB for every guest it runs. CPUID and MSR firewalls, interrupt injection, and the hypercall door all matter. None of them matter if EPT can map the same host frame to two guests without anyone noticing. We therefore name *exclusivity among guests* as the property we prove, not “the guest seemed isolated in tests,” and not “the hypervisor never touches a guest frame.”

2.2 In scope

We assume:

- **A malicious or compromised guest** (ring-0 Linux, firmware, or installer) that may issue any architected VM exit: EPT violation, CPUID, MSR, I/O, hypercall, HLT, exception.
- **Buggy device emulation** outside the Proven Core (virtio, IDE/ATAPI, UART). Bugs there can DoS a guest or confuse an installer; they must not become a second EPT mapping of a Proven Core frame.
- **Operator error** (wrong ISO, exhausted USB stick). Audit events record what happened; they do not enlarge the theorem.

Access versus mapping (virtio). Isolon ships virtio-blk and virtio-net. Virtqueue rings live in guest-allocated frames. Hypervisor emulation at VMX root *must* read and write those frames. That is, by construction, a guest-owned frame

touched by the hypervisor. Exclusive ownership in this paper is a statement about *EPT mappings*: a frame in *owned* has exactly one guest as EPT owner, and the hypervisor does not install a second EPT entry for that frame in another guest’s EPT. Transient, emulation-mediated access from VMX root is not a second owner. If “belongs to neither the hypervisor” is read as “the hypervisor CPU never loads from that frame,” the property is false of the product we ship. We do not claim that reading.

DMA and passthrough. The adversary in this section is a malicious ring-0 guest. Such a guest can program DMA if a device is passed through, or if an emulated device can be driven to DMA outside the guest’s frames. Isolon’s product path has **no device passthrough**. Guest DMA is mediated by emulation executing in the hypervisor’s address space. Exclusive ownership is *conditional on that*: it is not a theorem against an IOMMU-less passthrough device, and unmodelled DMA is listed in Section 2.3 rather than waved past. Closing DMA as a lemma needs an IOMMU story we do not have.

We do *not* assume a correct third-party guest firmware. Forcing a foreign firmware’s WaitForEvent internals to skip a check produced a NULL-event page fault and a dead loop. Product ISO boot now uses firmware we author. That firmware is **outside** the Proven Core.

2.3 Assumptions, not theorems

- The Intel SDM’s VMX/EPT hardware behaves as documented [10].
- The frozen Verus/SMT stack is trusted for the ghost proofs.
- Host physical memory given to the allocator is the memory the machine has.
- Hypervisor frames are those *absent* from the guest ownership registry. We do not prove that the EFI’s own page tables never map a guest-owned frame, and we do not prove that a bug cannot install a guest leaf pointing at hypervisor .text.
- No device passthrough; guest DMA is emulation-mediated, as above.
- Nested VT-x on a lab server is not a proof about a PowerEdge R640.
- Full x86 ISA / VMCS host-state correctness is inside the Proven Core *budget* and is not at proof-complete maturity beside the EPT crate.

3 Isolon Architecture

3.1 Single binary

Everything compiles to one UEFI PE/COFF image. Kernel, initrd, Web UI, and schemas are compressed sections, decompressed on first use. The size target is 15 MB with a 20 MB hard limit. There is no host Linux, no package manager, and no config tree the operator must get right before the first serial line.

Table 1. Proven Core budget (cap, not an L3 claim per row).

Module	Est. LOC	Why inside
VMX lifecycle	800	VMXON/VMXOFF leave CPU mode
VMCS management	1,500	Host-state corruption
EPT engine	2,000	Isolation mechanism
Frame allocator	1,500	Double-alloc / UAF
vCPU state	1,000	Save/restore leaks
Interrupt injection	800	Guest privilege
Hypercall door	500	Only intentional channel
MSR/CPUID/CR firewalls	1,200	Host feature subversion
Audit-log integrity	600	Tamper-evident trail
IPI confinement	500	Cross-VM IPI
Cap (code + scaffolding)	15,000	—

3.2 Proven Core boundary

Only the modules in Table 1 are in scope for Verus specifications and progressive proofs. Everything else uses Rust’s type system, testing, and review. Promotion into the Proven Core requires a written architecture decision; the default answer is no. The hard cap is 15,000 lines including proof scaffolding.

Lived split, as of this submission: the machine-checked exclusivity theorems live in a host-only crate (`ept_model`, 2,366 lines) that is *not* linked into the `.efi`. The executable EPT ownership registry is 705 lines at spec-written maturity. Device emulation, HTTP, the SPA, ISO, guest firmware, iDRAC, and migration tooling are outside.

3.3 Hardware

Dell PowerEdge R640 / R650 / R660. Low-risk, self-sufficient surfaces (serial via iDRAC virtual console, SMBIOS/ACPI topology, onboard LOM) are in scope. OEM RAID health schemas are not a blocker for the theorems or for the single-host product loop.

Code in the Proven Core is required to document invariants at the function boundary, keep unsafe blocks minimal, and tag each of them for bounded checking. Mutable globals require a lock and a justification comment. We use explicit state machines rather than boolean flags. These rules are how a codebase stays L3-shaped when most modules are still L1: the later proof should not require a rewrite of ownership.

3.4 Maturity ladder as a community instrument

We use four levels, kept even after a proof lands (runtime asserts stay):

- **L0** documented invariants in comments.
- **L1** runtime `assert! / debug_assert!` plus Kani on unsafe.
- **L2** Verus specifications with ghost state and contracts.
- **L3** cargo `verus verify` succeeds for the claimed surface.

Fallback is Verus $\rightarrow$ Kani $\rightarrow$ asserts and fuzz. The architecture always aims at L3; shipping is allowed at L1/L2 when L3 is still open, provided the paper does not upgrade the label.

We offer the ladder as a *proposal to the community*, not as Isolon process. A verification paper should be able to point at each of its claims and name the column. “We proved isolation” with an L3 ghost model, an L2 executable registry, and an L1 iron boot is three papers’ worth of sentences; fusing them is how proxy theorems get into abstracts. Requiring the column is cheap, checkable, and would have caught our own drafts.

Proven Core modules follow a four-file layout: executable Rust, a spec file, a proof file, and tests (Kani + unit). The exclusivity theorems of record do not use that layout. They live in one `verus!` block in `ept_model`, verified with cargo `verus verify`. Putting them beside live memory/`ept.rs` would imply the EFI crate is L3. It is not.

4 Exclusive Ownership

4.1 Statement

For every valid EPT mapping from a guest-physical address to a host-physical frame, that frame is owned by exactly one guest: no other guest holds a mapping to it.

“Exclusively owned” means exactly one guest holds a mapping to that frame at any moment. The property must hold across map, unmap, EPT-violation handling, and live-migration page transfer.

Hypervisor separation—the informal sentence “belongs to neither the hypervisor nor any other guest”—is *not* this theorem. Guest id 0 is reserved; hypervisor frames are modelled as absent from *owned*. Absence is definitional. We list it in Section 2.3 next to the SDM and the SMT stack. The ghost crate never mentions the EFI’s own page tables. A bug that installed a guest leaf pointing at `.text` would not be caught by these lemmas.

4.2 Ghost state

The ghost model is two finite maps (Figure 1):

- $owned : FrameId \rightarrow GuestId$
- $by_gpa : (GuestId \times Gpa) \rightarrow FrameId$

Guest id 0 is reserved (hypervisor / unmapped). 4K steps require page-aligned GPAs. A map step is enabled only when the frame is free and the (guest, GPA) pair is free. An unmap step is enabled only when that pair is present.

The predicate (from the crate, names only) is:

`exclusive_ownership(m)` iff every owned frame has a `by_gpa` witness, every `by_gpa` entry has a matching owner, a frame is the image of at most one pair, and the two maps have equal length.

Empty maps satisfy it. A hypervisor frame is modelled as *not* being in *owned*: the ghost registry only tracks guest-held frames.

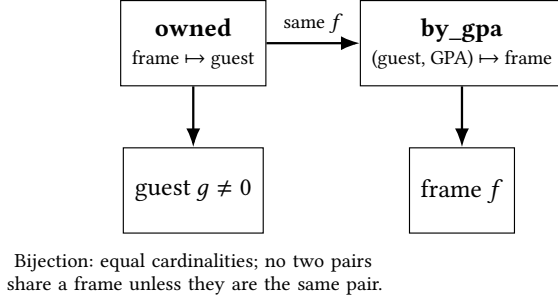


Figure 1. Ghost EPT ownership registry. Exclusive ownership is four conjuncts: every owned frame has a matching by_gpa witness; every by_gpa entry has a matching owner; a frame is the image of at most one pair; the two maps have equal length.

4.3 Steps

A 4K step is $\text{Map}\{g, a, f\}$ or $\text{Unmap}\{g, a\}$. Enabled steps preserve exclusive ownership (one-step lemma). Any finite sequence of enabled steps preserves it. Guest ids are already a field of each step, so the sequence theorem is already an N -guest statement. A second theorem, `n_guest_4k_map_unmap`, is the same induction under a CI-facing name; it does not quantify over guests in any way the first theorem does not. We do not count it as a distinct result. A two-guest lemma maps two distinct non-zero guests onto distinct free frames—that lemma is distinct: it instantiates two concrete ids.

Large pages are spans of 4K frames (512 for 2M, 2^{18} for 1G). Span map/unmap lemmas lift exclusivity to 2M/1G; unit lemmas record that 4K is a one-frame span. NUMA lemmas couple a ghost SRAT/SLIT topology with map/unmap so a mapped frame is recorded on its node. Hardware 2M-leaf decode lemmas relate a leaf PTE encoding to an identity-map intent. None of these lemmas are a full multi-level EPT walk of the EFI.

4.4 EPT-violation dispositions

An EPT miss is modelled as one of three dispositions:

- **EmulateNoMap** — emulate MMIO (APIC / virtio) without installing a mapping. Registry unchanged.
- **Reject** — fail closed. Registry unchanged.
- **ClaimMap** $\{g, a, f\}$ — demand-fill a 4K map, enabled only when that map step would be enabled.

Every enabled disposition preserves exclusive ownership. Emulate-then-claim is a derived post.

4.5 Page transfer

Live migration of a frame is unmap-at-source then map-the-same-HPA at destination. The step is enabled when source holds the GPA, destination GPA is free, and guests are distinct and non-zero. Posts: destination owns the frame at the new

GPA; the source mapping is gone; exclusivity holds. This is a ghost primitive. It is not vCenter vMotion, not a multi-host protocol, and not a proof that the product migrate engine is correct. That engine is outside the Proven Core.

4.6 Theorems of record

Table 2 lists the named theorems. Supporting lemmas (empty-exclusive, map-already-owned-unchanged, span preservation, mock bring-up facts) are in the same crate. CI counts 80 verified functions, including the N -guest alias. That number is the Verus obligation tally against the pin; it is not a claim that 80 distinct ideas were hard.

4.7 Proof sketch: map preserves exclusivity

We sketch the 4K map case; unmap is the inverse bookkeeping. Assume $\text{exclusive_ownership}(m)$, $g \neq 0$, a 4K-aligned, $f \notin \text{dom}(\text{owned})$, and $(g, a) \notin \text{dom}(\text{by_gpa})$. Let $m' = m.\text{ghost_map}(g, a, f)$.

Witness. For the new frame f , the pair (g, a) is the witness required by conjunct 1. Old frames keep their old witnesses because we did not delete by_gpa entries.

Owner match. The new by_gpa entry points at f , and $\text{owned}'[f] = g$. Old pairs are unchanged.

Uniqueness. Suppose two pairs in m' share a frame. If that frame is f , both pairs must be (g, a) because f was free. If not, uniqueness in m applies.

Cardinalities. Both maps gain one key, so lengths stay equal.

Verus discharges this with map lemmas from `vstd` (broadcast `group_map_lemmas` / `group_set_lemmas`). The one-step lemma case-splits on Map vs Unmap. The sequence theorem is induction on Seq length: the empty sequence is trivial; the inductive step applies the one-step lemma and recurses. The N -guest CI name is that same induction.

This 4K case is a data-structures exercise. We do not pretend otherwise. The modelling that was not free is below.

4.8 What the obvious formulation gets wrong

2M/1G spans. The obvious encoding of a 2M map is a loop of 512 4K maps. That is the wrong object. SMT would need a loop invariant over 512 iterations, still would not know that those 512 keys are *one* leaf with the page-size bit, and would still be 512 ghost steps rather than one hardware PDE. What we wrote instead is a span predicate (every aligned 4K key in the region is free) followed by a bulk insert, plus an arithmetic lemma that 512 aligned 4K frames constitute a 2M leaf (2^{18} for 1G). Those facts are still not a hardware page-walk. They are why “map 4K k times” is not the 2M theorem.

Violation dispositions. The obvious miss handler is demand-fill: every EPT violation becomes ClaimMap. That formulation installs a mapping for APIC and virtio MMIO GPAs, either aliasing device memory into the ownership

Table 2. Named exclusivity theorems in the host-only crate.

Theorem	Claim
single_guest_4k_map_unmap	finite 4K sequences
n_guest_4k_map_unmap	CI alias of the previous row
concrete*_guest_refine	scoped ghost $\leftrightarrow$ exec
large_page_map_unmap	2M/1G spans
numa_map_unmap_affinity	node affinity ghost
hw_2m_leaf_refines_	2M leaf encode
ept_violation_preserves	miss dispositions
page_transfer_preserves	unmap $\rightarrow$ map handoff
alloc_map_unmap_refines	pool + EPT refine

registry or handing the guest a host frame for a region that should stay unmapped. The trichotomy (EmulateNoMap / Reject / ClaimMap) is the fix: MMIO is emulated with the registry unchanged; fail-closed is identity on the maps; demand-fill is enabled only when the 4K map step would be. Emulate-then-claim is a derived post.

NUMA affinity. Exclusivity over *owned* and *by_gpa* cannot see sockets. A mapped frame on the wrong node is still exclusive. Coupling a ghost SRAT/SLIT so that a successful map is recorded on its node is extra state; without it, affinity is a comment.

No bug-finding story from SMT. We did not catch an executable aliasing bug with Verus or Kani. The ghost model was written against a registry that already rejected double-map of the same HPA. Kani’s bounded harnesses (no_double_map_same_hpa, allocator no-alias) pass; they did not surprise us with a counterexample. Runtime on iron *did* find real bugs—Enable-XSAVES, INVPCID, WaitForEvent state pokes (Table 6)—none of which are exclusivity lemmas. A verification paper with no SMT-found bug and no refinement to the binary has a thin claim; we prefer that sentence to a reconstructed catch.

Violation ClaimMap reuses the 4K map lemma. Emulate and reject are identity on the registry; the proof is $m' = m$. Transfer is unmap then map of the *same* frame: after unmap, f is free, so the destination map is enabled. Posts are the destination owner and the absent source key.

5 The Ghost / Executable Gap

A discharged ghost model is not a proved .efi. Isolon records that as a first-class result, not a footnote.

Live EPT (memory/ept.rs, 705 lines) maintains an ownership registry and asserts exclusive-ownership-shaped facts at map/unmap. Beside it sit 357 lines of spec/proof notes. Those notes are not the 80-function crate. The host-only crate exists *because* linking Verus into the UEFI target is the wrong packaging: proofs run on a Linux CI host under a pinned Verus release; the EFI is a UEFI target, not a Linux sysroot.

Scoped refinement lemmas show that a concrete map representation, under a restricted interpretation, commutes with ghost map/unmap for 4K (one guest, then N guests) and that an allocate-then-map path refines. They are not a bisimulation of VMXON, VMCS host state, EPT pointer load, and every MMIO emulator the EFI contains.

Consequences we accept in print:

- L3 is the ghost crate. Live EPT is L2.
- VMX, VMCS, hypercalls, MSR firewalls, audit integrity, and IPI confinement are in the budget in Table 1 and are not at L3 beside ept_model.
- Iron serial logs raise confidence in the *runtime path*. They do not raise Verus maturity. Lab L3 and R640 L1 are both real; they are not the same claim.

5.1 Live registry versus ghost maps

The live engine is not a Verus Map. It is a bounded table: EptMap holds at most 512 explicit 4K claims in production builds (8 under Kani, so CBMC can unwind), plus a 16-slot range registry for the precise identity window used at Linux bring-up.

Collision and exhaustion are different errors. Map or range-claim of a taken frame or overlapping span returns AlreadyOwned. When every slot is occupied, both tables return Full. There is no silent drop and no fallback from 4K slots into range claims.

Linux bring-up does not fill the 512-slot 4K table. Guest e820 is 256 MiB; the precise identity window is $[0, \text{PRECISE_BYTES})$ with $\text{PRECISE_BYTES} = 512$ MiB (131,072 4K pages). Hardware identity uses 2M leaves (256 leaves). The executable counterpart of that window is the range registry: with no extra-guest holes, bring-up inserts **one** range covering the full 512 MiB for the Linux guest. Multi-VM probes punch private HPA holes and claim those spans for shell guests, still as range rows, not 4K rows. The 4K table is the demand-fill and self-test path (map a bring-up guest, then assert a second guest cannot alias the same frame). Scoped refinement lemmas interpret that 4K table into ghost maps. They do not interpret the range registry that actually covers Linux’s 512 MiB. If a 513th distinct 4K claim is required, the live table returns Full while the ghost theorem still holds of an abstract ghost_map. That mismatch is why we will not stamp L3 on memory/ept.rs from the 80/0 line.

The honest framing is that the ghost model corresponds to a demand-fill path currently exercised by the self-test and by MMIO-not-mapped misses that take ClaimMap, while production Linux GPA coverage is a range claim plus 2M hardware leaves. Correspondence is scoped (hardware 2M-leaf decode lemma + range self-test), not “every live PTE has a ghost key.”

5.2 What would close the gap

Three artifacts, none of which this paper claims:

1. A proved executable EPT walker whose leaves are in 1–1 correspondence with `by_gpa`, including the HV-vs-guest split (“belongs to neither” as a fact about HV page tables, not as absence from a guest registry).
2. An IOMMU / DMA story: devices we do not model cannot write a guest-owned frame or a hypervisor frame.
3. Linking or replaying (1) on the UEFI target, or a refinement from the EFI binary’s EPT root pointer to the ghost maps.

Until then, “memory isolation isn’t tested; the ghost exclusivity theorems are proved” is the strongest true slogan. “The hypervisor is proved” is false.

6 Evaluation

We evaluate four questions. (1) Does the ghost model actually verify? (2) How large is the proved surface relative to the hypervisor and the size budget? (3) What covers guest-physical space at Linux boot? (4) What does the runtime do on a lab host versus a real R640?

6.1 Proofs

Toolchain: Verus pin 0.2026.07.12 (asset 0b42f4c; Linux zip SHA-256 f6f4f5d0. . . fbd01c0 in-tree; CI never uses latest) and Kani 0.67.0 (CBMC backend). The exclusivity crate is verified with `cargo verus verify`. Against that pin the crate reports **80 verified, 0 errors**, including the *N*-guest alias, with no admit on the exclusivity path. We did not record a wall-clock or peak RSS for that run on named hardware in this draft; the artifact is the crate plus the pin (supplement). Kani covers bounded unsafe in the live engine (MAP_CAP = 8 unwind). That is L1 on unsafe, not L3 on exclusive ownership.

6.2 Size, occupancy, and what we did not measure

Table 3 is tree- and kit-measured. The iron EFI that reached Linux shell plus the M4 probe chain is 1,085,952 bytes (1.04 MiB), against a 15 MB target and a 20 MB hard limit. A later in-tree release build is 1.62 MiB; both are well under the cap. Rust sources in the tree total 64,140 lines (wc -l of *.rs, excluding target/): about 40,500 product, 15,600 gate harnesses, 8,000 tests. The Proven Core *budget* remains 15,000 lines (Table 1); the surface actually at L3 is the 2,366-line host-only crate. Table 4 is the EPT slice of that.

Linux occupancy is from the bring-up path, not from a COM2 dump of slot counters. One range covers 512 MiB; hardware uses 256×2M leaves; guest e820 is 256 MiB. The 512-slot 4K table is not that window.

We do not have VM-exit round-trip cycles (CPUID, EPT violation, HLT) versus KVM on the same R640, and we do not have a timed boot-to-shell versus KVM. COM2 archives

Table 3. Measured quantities. Cycle-level KVM comparison is absent, not estimated.

Quantity	Value
Iron .efi (Linux+M4 boot)	1,085,952 B (1.04 MiB)
In-tree release .efi	1.62 MiB
Size target / hard limit	15 MB / 20 MB
Tree Rust LOC (*.rs)	64,140
Proven Core budget / L3 crate	15,000 / 2,366
Verus pin zip SHA-256	f6f4f5d0. . . fbd01c0
Verus functions (incl. alias)	80 verified, 0 errors
Linux identity window	1 range, 512 MiB
Hardware 2M leaves in that window	256 (131,072 4K pages)
Guest e820 RAM	256 MiB
4K table cap / range slots	512 / 16; exhaustion = Full
VM-exit cycles vs KVM on R640	not measured
Boot-to-shell wall-clock vs KVM	not measured
verus verify wall-clock / RSS	not recorded for this PDF

Table 4. Line counts (wc -l) for the EPT-related surface.

Path	Lines	Role
ept_model crate	2,366	L3 ghost + proofs
memory/ept.rs	705	live EPT (L2)
ept_spec + ept_proof	357	notes beside live
frame allocator + spec	362	allocator + ghost set

of the iron boot have no usable timestamps for that interval. We will not invent those numbers.

6.3 Runtime: lab host versus R640

Table 5 is the split. Cells are contemporaneous serial or host-test evidence, not a wish list. Nested VT-x is a third column because operators (and reviewers) otherwise collapse it into iron.

On the R640 (iDRAC virtual media, COM2, Intel Xeon Silver 4110): the EFI archives VMXON, a 512 MiB identity window, unmodified Linux 6.12 through /init to a shell, then—on that same boot—four-VM probes (Linux G0 plus three SHELL guests), virtio-blk, virtio-net, dual-vCPU SMP, and VMXOFF. Secondary Enable-XSAVES was required on Xeon Scalable; without it the guest init hit a stack guard. Firmware SNP/Tcp4 after ExitBootServices is a platform limit of the virtual-floppy path; durable HTTP is a host-owned Broadcom LOM. The SPA can VMLAUNCH a private-EPT SHELL guest and re-enter; that guest is a CPUID stub, not a distro. Guest firmware issued ATAPI READ(10) and booted El Torito CD EFI. Persist-detect stamps on a USB stick are LBAs, not a guest root filesystem.

On nested VT-x we authored guest firmware after third-party WaitForEvent forcing faulted. That firmware completed an alpine-extended setup-disk -m sys (installer

Table 5. Split evaluation. Nested VT-x $\neq$ PowerEdge. Host Verus $\neq$ EFI.

Surface	Lab / CI	Nested VT-x	PowerEdge R640
Verus exclusivity (80 / 0)	yes (L3)	—	not claimed
Kani on unsafe	yes (CI pin)	—	not claimed
VMXON + VMEXIT	yes	yes	yes
EPT identity + Linux 6.12 shell	yes	—	yes (Xeon Silver 4110)
LAPIC timer re-entry / IRQ inject	yes	—	yes (same boot)
virtio-blk probe	yes	CD+virtio	yes (same boot)
virtio-net + vSwitch probe	yes	—	yes (same boot)
Dual-vCPU BSP+AP (shared EPT)	yes	—	yes (same boot)
Four VMs: Linux + three SHELL stubs	yes	—	yes (same boot)
Durable host-NIC HTTP after EBS	lab NIC	—	onboard LOM
SPA launches a SHELL guest	host smoke	—	stub, not a distro
Guest firmware CD EFI (El Torito)	—	CD+virtio	yes
Distro sysinstall + reboot-to-disk	—	yes	open
Persist-detect LBA stamps on USB	—	—	stamps, not a rootfs

printed “Please reboot”) and reboot-to-disk. Nested virtualization is not the R640. The USB stick used on iron is too small for that distro image. We do not print an install-complete marker from host CI or from nested runs.

6.4 What we refuse to count as a theorem

72-hour soak metrics on a laptop, PDF audit-report templates, and a bring-up spec review are operational gates. They closed on the lab host. They are L0/L1 process, not L3. Iron distro install is a product gate. It is not required for the ghost theorem; claiming it would be a correctness reject.

6.5 Artifact

We commit to a supplementary zip: the `ept_model` crate, the pinned Verus zip SHA-256, a README that reproduces 80/0, and a container or exact install command for that pin. Anonymized COM2 excerpts for every row of Table 5’s R640 column (product markers stripped) belong in the same zip. The paper stands alone if the reviewer ignores the zip.

7 Experience: Five Lessons that Generalized

EuroSys invites experience papers whose lessons are general, supported by rigorous and quantitative analysis, and useful to the community [1]. The following are the ones we would tell a group starting a verified Type-1 hypervisor in Rust. They are the spine of the paper. The ghost model is an artifact inside this story.

7.1 Pin the prover or the proof is a diary

Verus and its SMT backend move. We froze the tagged Verus release 0.2026.07.12 (asset 0b42f4c) with a SHA-256 of the Linux zip in-tree, and forbade CI from fetching latest. The 80/0 line is replayable only against that pin. Groups that verify “on nightly” will not be able to say, a quarter later,

whether a new `admit` is a regression or a toolchain change. Budget pin maintenance as a first-class task, not as hope.

7.2 The UEFI target cannot run SMT

Isolon’s EFI target is UEFI PE/COFF, not a Linux `sysroot`. Verus wants a host `sysroot`. We stopped trying to link proofs into the PE/COFF image. The 2,366-line crate lives on the CI host; the 705-line engine lives in the EFI. That split is why L3 and L2 are different columns in Table 5. A single-crate “verified hypervisor” that does not build on the target you ship will still be described as verified by someone; make the packaging match the claim first.

7.3 Do not poke third-party firmware

When guest firmware blocked in `WaitForEvent`, a tempting fix is to poke its wait object, skip a check, or inject a fault. We did that on a development path. The firmware then page-faulted on a NULL event pointer and entered a dead loop. We do not own that firmware’s invariants; forcing one of them violates the next. The product path is firmware we author, driven only through architected inputs (interrupts, device registers, DMA the spec says is ours). Device emulation remains outside the Proven Core, so a firmware bug is still not an EPT exclusivity theorem—but it is no longer a self-inflicted one. This lesson is independent of Isolon: if you do not own the blob, do not rewrite its control state.

7.4 Firmware TCP is not a management plane

iDRAC virtual floppy advertised PXE/HTTP but not a `Tcp4` service binding. Firmware SNP that had accepted a listen before `ExitBootServices` did not survive it. Durable HTTP on the R640 is a host-owned Broadcom LOM after the hypervisor owns the NIC, not a firmware socket. Quantitative split: PRE-EBS HTTP closed on SNP as a bring-up window; post-EBS HTTP closed only after a native driver. Papers that claim “the hypervisor serves a Web UI” should say which

Table 6. Archived negative results. None of these are L3.

Failure	What it blocked
Guest /init stack guard without Enable-XSAVES	Linux shell on Xeon Scalable
Firmware Tcp4 absent on virtual floppy	“HTTP forever” via SNP
SNP listen dead after ExitBootServices	post-EBS firmware UI
WaitForEvent state poke → #PF	third-party firmware as product ISO path
USB persist LBA stamps	calling stamps a root filesystem
USB stick capacity < alpine-extended	iron sysinstall on that stick

NIC stack and which side of ExitBootServices. In-process REST on a lab executable is a third, still weaker, claim.

7.5 Nested VT-x is not the production server

Nested alpine-extended sysinstall completed: installer printed “Please reboot,” then reboot-to-disk succeeded. That is a real product-path result. It is also the result most likely to be copy-pasted onto the R640 column. We refuse that paste. The USB stick used on iron is too small for that distro image; El Torito CD EFI is not setup-disk. A lab hypervisor that installs Linux under nested VT-x has not demonstrated ISO install on the machine in the rack. Table 5 exists so we cannot “fix” that with prose.

7.6 Negative results we archived

Table 6 lists failures we kept. They are not theorems. They stopped us from claiming theorems we did not have.

The XSAVES miss is the most transferable: a VMCS secondary-control bit that is optional on a laptop CPU and required on the server. Isolation proofs do not mention it. The guest still does not boot. Treat CPU-feature bring-up as L1 evidence, not as a corollary of exclusive ownership.

8 Related Work

Proved kernels and enclaves. seL4 [12, 13, 20] is the landmark machine-checked microkernel: functional correctness, then integrity and information flow. Isolon borrows the discipline—small trusted core, explicit invariants, proofs against a written theorem—and refuses seL4’s scope. We do not prove the scheduler or the device model. CertiKOS [4, 7] builds a certified concurrent kernel with contextual refinement. Komodo [6] verifies a software enclave monitor on ARM TrustZone. Ironclad Apps [8] push verification out to application principals. These systems show isolation *can* be proved. They do not ship as BOOTX64. EFI on iDRAC virtual media.

Hypervisors with a verification story. Hyperkernel [21] pushes a hypervisor toward push-button verification with a

simplified interface. XMHF [25] is an extensible hypervisor framework with selected proofs. Verisoft XT applied VCC to Hyper-V components [17]. SeKVM / MicroV work [18, 19] layers formal memory protection onto a commodity KVM. Isolon is the opposite packaging: a new UEFI binary with a tiny proved ghost registry, not a verified overlay on Linux. We do not claim a larger TCB proof than SeKVM; we claim a smaller theorem that we have actually discharged in Verus, plus honest labels on everything else.

Small-TCB virtualization. NOVA [23] and related microhypervisor designs argue for a small TCB in the EuroSys tradition. Isolon’s cut is the Proven Core list in Table 1, not a second kernel in the guest. OKL4-style microvisors [9] similarly shrink the host. Production hypervisors—ESXi [3], Xen [2], KVM [11]—isolate with EPT/NPT and years of testing. We do not compete on feature count. We publish the gaps (live EPT at L2, iron installer open) instead of implying the theorem covers the SPA.

Tooling. Verus [15, 16] is the SMT prover we pin. Kani [5, 14] covers unsafe; it compiles MIR to CBMC. VanHattum et al. [24] is the ICSE-SEIP paper on dynamic trait objects in that pipeline, not a substitute for the tool or for CBMC. Push-button verification of file systems [22] is the methodological cousin of Hyperkernel; our exclusivity proofs are lemma-heavy rather than fully push-button across the EFI.

9 Limitations

- Live executable EPT is L2. L3 is the host-only ghost model. Ghost $\leftrightarrow$ exec refine is scoped, not a full EFI bisimulation.
- Most Proven Core modules in Table 1 are not at L3.
- Large-page and NUMA L3 are lab/CI. Iron stopped at multi-VM probes for those surfaces.
- PageTransferStep is a ghost primitive, not live migration product.
- DMA confinement and IOMMU are future work. Exclusive ownership is conditional on no passthrough and emulation-mediated DMA.
- Nested distro sysinstall is complete and is not iron.
- Iron installer, TLS, and guest console remain product residuals.
- Verus upgrades can break proofs; we budget pin maintenance and refuse latest.
- SMT found no executable aliasing bug. We do not reconstruct one.
- We have no KVM cycle comparison on the R640.

Threats to validity. Iron evidence is one PowerEdge R640 and iDRAC virtual media, not a fleet. Nested VT-x is one lab host. The 80/0 line is one pin of one prover; we have not replayed under a second SMT solver. LOC counts are wc -l, not a proven TCB. Registry occupancy is from the bring-up code path, not from a per-boot slot dump on COM2.

Table 7. Where Isolon sits. “Proved” means a machine-checked theorem of the named property, not “tests passed.”

System	What is proved	Guest / boot	Isolation property	This paper
seL4	kernel func. + integrity/IFC	user tasks, not EFI HV	info-flow / integrity	discipline, not scope
CertiKOS	concurrent kernel refine	certified OS	contextual refine	not our TCB cut
Komodo	enclave monitor	TrustZone	enclave isolation	different hardware
Hyperkernel	push-button kernel	simplified HV iface	interface properties	we keep full VMX/EPT
SeKVM	KVM memory protection	Linux+KVM	hyp. memory protection	overlay vs clean-slate
NOVA	(design) small TCB	microhypervisor	TCB size argument	our cut is Proven Core
Xen/KVM/ESXi	—	production guests	tested EPT/NPT	we claim a smaller theorem
Isolon	guest-exclusivity (ghost maps)	EFI .efi, Linux on R640	guest EPT exclusivity, not HV access	L3 ghost, L2 live EPT

Experience lessons (XSAVES, SNP, firmware pokes) are one implementation’s failures; we argue they generalize because they follow from Intel VMX, EFI ExitBootServices, and “do not own that blob,” not from Isolon-specific APIs. We may be wrong about that generalization.

10 Conclusion

Isolon is an experience report about keeping a verification claim honest while shipping a EFI Type-1 hypervisor. The maturity ladder is the instrument: L3 for a guest-exclusivity ghost model, L2 for the live registry, L1 for the R640 runtime. The theorem is exclusivity among guests, not hypervisor non-access, and not the EFI. Virtio rings are guest frames the emulator touches; the property is about mappings. DMA is out unless passthrough stays off.

On iron: a PowerEdge R640 printed VMX, a 512 MiB identity window, Linux 6.12 to a shell, virtio-blk and virtio-net, SMP, four-VM probes, host-NIC HTTP, a SHELL guest from the SPA, and El Torito CD EFI. Nested authored firmware completed a distro sysinstall. Nested is not iron.

Lead with the claim: a hypervisor you can boot, a ghost model you can replay, and a ladder that forbids fusing those two sentences. The product loop is not done. That last fact is residual, not the title.

References

- [1] 2026. EuroSys 2027 Call for Papers. <https://2027.eurosys.org/cfp.html>. Experience papers: lessons that are general, supported by rigorous and quantitative analysis.
- [2] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. 2003. Xen and the Art of Virtualization. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. 164–177. <https://doi.org/10.1145/945445.945462>
- [3] Edouard Bugnion, Scott Devine, Mendel Rosenblum, Jeremy Sugerman, and Edward M. Bugnion. 2012. Bringing Virtualization to the x86 Architecture with the Original VMware Workstation. *ACM Transactions on Computer Systems* 30, 4 (2012), 12:1–12:51. <https://doi.org/10.1145/2384592.2384593>
- [4] David Costanzo, Zhong Shao, and Ronghui Gu. 2016. End-to-End Verification of Information-Flow Security for C and Assembly Programs. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 648–664. <https://doi.org/10.1145/2908080.2908100>
- [5] Rémi Delmas, Ziad Hassan, Qinheping Hu, Rahul Kumar, Felipe R. Monteiro, Thanh Nguyen, Adrián Palacios, Celina Val, Michael Tautschnig, Justus Adam, Daniel Schwartz-Narbonne, and Carolyn Zech. 2026. Kani: A Model Checker for Rust. *arXiv:2607.01504*. To appear, 39th IEEE/ACM International Conference on Automated Software Engineering (ASE).
- [6] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. 2017. Komodo: Using Verification to Disentangle Secure-Enclave Hardware from Software. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*. 287–305. <https://doi.org/10.1145/3132747.3132782>
- [7] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 653–669.
- [8] Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. 2013. Ironclad Apps: End-to-End Security via Automated Full-System Verification. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 165–181.
- [9] Gernot Heiser and Ben Leslie. 2010. The OKL4 Microvisor: Convergence Point of Microkernels and Hypervisors. In *Proceedings of the First ACM Asia-Pacific Workshop on Systems (APSys)*. 19–24. <https://doi.org/10.1145/1851276.1851282>
- [10] Intel Corporation. 2024. Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 3: System Programming Guide. (2024). Order number 325384. Extended page tables (EPT) and VMX.
- [11] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. 2007. KVM: The Linux Virtual Machine Monitor. In *Proceedings of the Linux Symposium*. 225–230.
- [12] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. 2014. Comprehensive Formal Verification of an OS Microkernel. *ACM Transactions on Computer Systems* 32, 1 (2014), 2:1–2:70. <https://doi.org/10.1145/2560537>
- [13] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*. 207–220. <https://doi.org/10.1145/1629575.1629596>
- [14] Daniel Kroening and Michael Tautschnig. 2014. CBMC – C Bounded Model Checker. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 389–391. https://doi.org/10.1007/978-3-642-54862-8_26

- [15] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. 2024. Verus: A Practical Foundation for Systems Verification. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*. 438–454. <https://doi.org/10.1145/3694715.3695952>
- [16] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proceedings of the ACM on Programming Languages*, Vol. 7. 85:1–85:30. <https://doi.org/10.1145/3586037>
- [17] Dirk Leinenbach and Thomas Santen. 2009. Verifying the Microsoft Hyper-V Hypervisor with VCC. In *Proceedings of the 2nd World Congress on Formal Methods (FM)*. 806–809. https://doi.org/10.1007/978-3-642-05089-5_51
- [18] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. Formally Verified Memory Protection for a Commodity Multiprocessor Hypervisor. In *Proceedings of the 30th USENIX Security Symposium*. 3953–3970.
- [19] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. A Secure and Formally Verified Linux KVM Hypervisor. In *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*. 1782–1799. <https://doi.org/10.1109/SP40001.2021.00049>
- [20] Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. 2013. seL4: From General Purpose to a Proof of Information Flow Enforcement. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy*. 415–429. <https://doi.org/10.1109/SP.2013.35>
- [21] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Hyperkernel: Push-Button Verification of an OS Kernel. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*. 252–269. <https://doi.org/10.1145/3132747.3132748>
- [22] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. 2016. Push-Button Verification of File Systems via Crash Refinement. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 1–16.
- [23] Udo Steinberg and Bernhard Kauer. 2010. NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*. 209–222. <https://doi.org/10.1145/1755913.1755935>
- [24] Alexa VanHattum, Daniel Schwartz-Narbonne, Nathan Chong, and Adrian Sampson. 2022. Verifying Dynamic Trait Objects in Rust. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 321–330. <https://doi.org/10.1145/3510457.3513039>
- [25] Amit Vasudevan, Sagar Chaki, Limin Jia, Jonathan McCune, James Newsome, and Anupam Datta. 2013. Design, Implementation and Verification of an Extensible and Modular Hypervisor Framework. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy*. 430–444. <https://doi.org/10.1109/SP.2013.36>